Фролов Иван Григорьевич, HW5

1. Статический анализ (SpotBugs + PMD)

Добавлены:

- стандартная конфигурация PMD и SpotBugs,
- задача `staticAnalyze`,
- генерация html/xml отчетов,
- задача `runProfilingLoadTest` для нагрузочного профилирования.

Первичный прогон

Выполнен запуск:

- `./gradlew clean staticAnalyze`

Отчеты сохранены в:

- `before_reports/pmd/main.html`
- `before_reports/spotbugs/main.html`

Исправления по замечаниям анализаторов

Исправлено в коде:

1. Утечка in-memory сессий при логауте:

- `src/main/ru/hse/server/AccountServer.java`
- при `logout` теперь удаляются записи из `activeSessions` и `sessionEnabled`, также обнуляется `account.activeSession`.

2. Ошибка проверки сессии и утечки `ResultSet`:

- `src/main/ru/hse/server/ServerStorage.java`
- `testSession` исправлен с `executeUpdate()` на `executeQuery()`.
- все `ResultSet` переведены на try-with-resources.
- добавлен `close()` для корректного завершения пула `HikariDataSource`.

3. Корректная остановка сервера:

- `src/main/ru/hse/server/Server.java`
- `stop()` теперь закрывает API сервер и хранилище БД (`storage.close()`).

4. Исправление затенения поля:

- `src/main/ru/hse/server/ApiServer.java`

- поле сервера переименовано в `accountServer`, обращения в обработчиках обновлены.

Повторный прогон

Выполнен повторный запуск:

- `./gradlew clean staticAnalyze`

Отчеты сохранены в:

- `after_reports/pmd/main.html`
- `after_reports/spotbugs/main.html`

---

2. Анализ кода и форматирование (VS Code)

Примечание: так как у меня нет OpenIDE, то я спользовал альтернативные плагины для VS Code

Инструмент	IntelliJ IDEA	VS Code эквивалент
Code Complexity	code-complexity-plugin	Metrics или Cognitive Complexity
Google Style Checkstyle	checkstyle-idea	Checkstyle for Java
Code Formatting	idea_plugin	Google Java Format

2.1 Когнитивная сложность (Metrics)

Server.java

UNCANNY COGNITIVE COMPLEXITY

Cognitive Complexity: 8

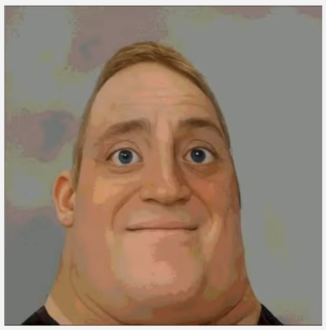
REPORT.md U ● | ServerProfilingLoadTest.java 9+, U | Server.java 9+, U × | README.m

hw5 > HomeTask05Java > src > main > ru > hse > server > Server.java > { } ru.hse.server

```java
package ru.hse.server;

import java.util.Scanner;
import java.util.concurrent.atomic.AtomicBoolean;

public class Server {
    ServerStorage storage = null;
    ServerLogicProxy storageProxy = null;
    AccountServer accountServer = null;
    ApiServer apiServer = null;
    AccountSecurity accountSecurity = null;

    public void start(int port) {
        storage = new ServerStorage("accounts");
        storageProxy = new ServerLogicProxy(storage);
        accountServer = new AccountServer(storage, storageProxy);
        // security server, tends to be active only when accounts are logged
        accountSecurity = new AccountSecurity(accountServer, storage);
        apiServer = new ApiServer(accountServer, port);

        storage.setChangeVerifier(accountSecurity);
        apiServer.addAuthListener(accountSecurity);
    }

    public void stop() {
        if (apiServer != null) {
            apiServer.stop();
        }
```

Account.java

UNCANNY COGNITIVE COMPLEXITY



Cognitive Complexity: 20

REPORT.md U ● | Account.java 9+, U × | ServerProfilingLoadTest.java 9+, U | READ

hw5 > HomeTask05Java > src > main > ru > hse > Account.java > ...

```java
package ru.hse;

import java.util.Random;

public class Account {
    private IAccountDataSource storage;
    protected final String login;
    private String[] privateKey = new String[256];
    public Long activeSession = null;

    public final String getLogin() {
        return login;
    }

    public final Long getActiveSession() {
        return activeSession;
    }

    public Account(String login) {
        this.login = login;
        Random r = new Random(login.hashCode());
        for(int i = 0;i<privateKey.length;i++)
            privateKey[i] = Long.toString(r.nextLong())+Double.toString(r.
    }

    public OperationResponse withdraw(double amount) {
```

`ServerStorage.java`

UNCANNY COGNITIVE COMPLEXITY

Cognitive Complexity: 52

REPORT.md U | ServerStorage.java 9+, U | ServerProfilingLoadTest.java 9+, U

hw5 > HomeTask05Java > src > main > ru > hse > server > ServerStorage.java > { } ru.hse.server

```java
package ru.hse.server;

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import ru.hse.IAccountDataSource;
import ru.hse.IAuthorizationSource;
import ru.hse.OperationResponse;

import java.io.IOException;
import java.io.InputStream;
import java.sql.*;
import java.util.Properties;

public class ServerStorage implements IAccountDataSource, IAuthorizationSou
    private HikariDataSource source;
    private final String filename;

    public ServerStorage(String filename) {
        this.filename = filename;
        setupDatabase();
        initDatabase();
    }

    private void setupDatabase() {
        String pass = "";
        String username = "SA";
        try (InputStream input =
```

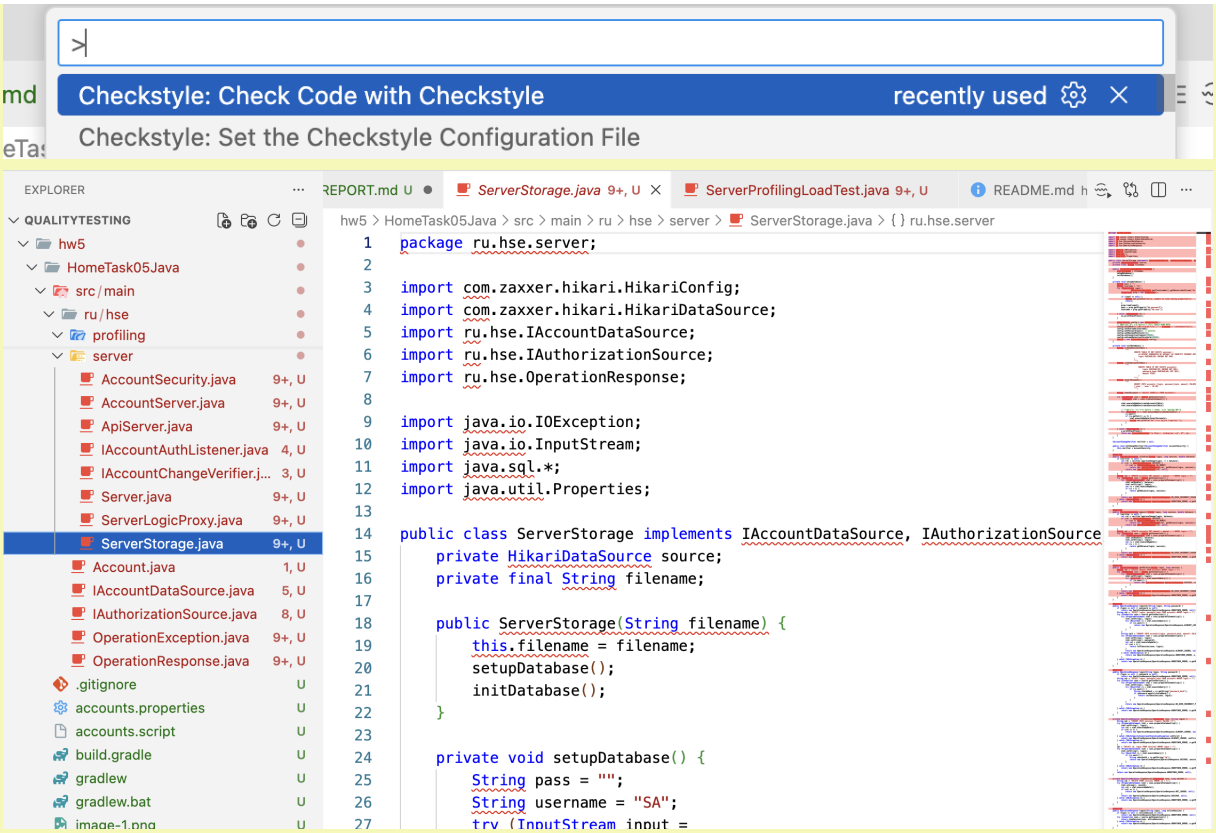
То есть, файл `ServerStorage` самый когнитивно сложный

К сожалению, это расширение может посмотреть только весь файл, а не каждую функцию отдельно

2.2 Google Java Style (Checkstyle + Google Java Format)

Этап 1: Проверка нарушений стиля (Checkstyle)

Checkstyle для файла `ServerStorage.java`

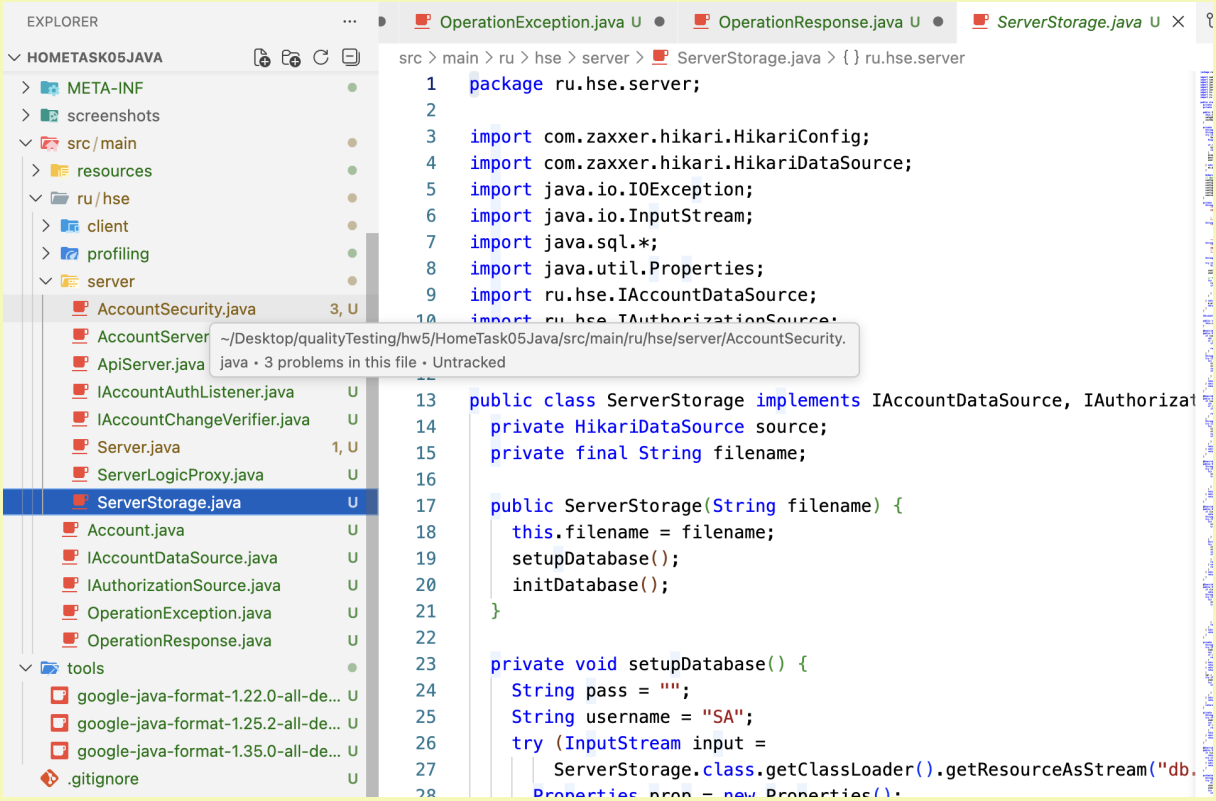


Все красное. Надеюсь, так и должно быть)

Используем именно Google Format

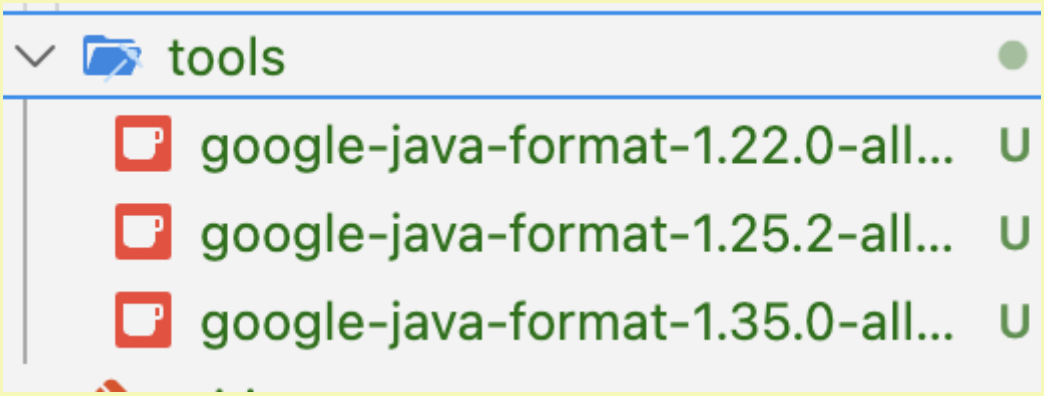
Этап 2: Форматирование кода

Плагин: Google Java Format



В общем это оказалось не так и просто
Файлы постоянно ругались

Пришлось установить 3 дополнительных файла для работы этого расширения



Потом все равно было много красных стрелочек, поэтому делал инструкцию от AI

1. Откройте **только** папку `HomeTask05Java` (`File → Open Folder...`).
2. Убедитесь, что установлены расширения:
   - `redhat.java`
   - `vscjava.vscode-gradle`
   - `vscjava.vscode-java-dependency`
3. Откройте Command Palette (`Cmd+Shift+P`) и по очереди:
   - `Java: Clean Java Language Server Workspace` → подтвердить restart
   - `Java: Import Java Projects`
   - `Gradle: Refresh Gradle Project` (или `Gradle: Refresh All`)
4. В панели **JAVA PROJECTS** должен появиться проект с исходниками из src/main и зависимостями из `build.gradle`.
5. Если не появился:
   - `Cmd+Shift+P` → `Developer: Reload Window`
   - снова `Java: Import Java Projects`.

Кажется, только этого все стало зеленым
Надеюсь, такой результат подходит

Но, если честно, сам до конца не уверен, что там поменялось

---

3. Профилирование

Добавлен класс:

- `src/main/ru/hse/profiling/ServerProfilingLoadTest.java`

Сценарий:

- регистрация аккаунта,
- депозит 100,
- снятие 200,
- снятие 50,
- logout,
- периодические неудачные попытки логина.

Поддерживает запуск с параметрами:

- `iterations` (по умолчанию 300000),
- `port` (по умолчанию 7001).

Пример запуска:

- `./gradlew runProfilingLoadTest --args='300000 7001'`

Вот что он вывел:

```
(.venv) admin@MacBook-Pro HomeTask05Java % ./gradlew runProfilingLoadTest --args='3
00000 7001'
termine if they come from your own scripts or plugins.

For more on this, please refer to https://docs.gradle.org/9.1.0/userguide/command_l
ine_interface.html#sec:command_line_warnings in the Gradle documentation.

BUILD SUCCESSFUL in 2m 49s
3 actionable tasks: 1 executed, 2 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.o
rg/9.1.0/userguide/configuration_cache_enabling.html
```

Как я понял профилировщик создал файл `profiling.jfr`

После запуска проанализировали heap

(запускал на более быстром тесте, так как исходный идет 2 минуты) Создался файл `profiler_heap_after_fix.txt`

```
Version: 2.1
Chunks: 1
Start: 2026-03-11 12:37:44 (UTC)
Duration: 8 s

Event Type                                Count  Size (bytes)
================================================================
jdk.GCPhaseParallel                        2725         67304
jdk.ModuleExport                           1372         14965
jdk.NativeLibrary                           878         76014
jdk.PromoteObjectInNewPLAB                  866         14672
jdk.ThreadPark                              741         25610
jdk.ThreadSleep                             638         12122
jdk.JavaExceptionThrow                      536         35864
jdk.ObjectAllocationSample                  513          7563
jdk.BooleanFlag                             483         14354
jdk.SystemProcess                           466         50789
jdk.ActiveSetting                           376          9324
jdk.NativeMethodSample                      361          3970
jdk.Checkpoint                              326       1919936
jdk.JavaErrorThrow                          204          8250
jdk.Deoptimization                          191          4546
jdk.ModuleRequire                           162          1620
```

Вот потенциальные утечки памяти

```
(.venv) admin@MacBook-Pro HomeTask05Java % jcmd | grep -i ServerProfilingLoadTest
45205 ru.hse.profiling.ServerProfilingLoadTest 300000 7001
(.venv) admin@MacBook-Pro HomeTask05Java %
```

В общем, надеюсь это примерно то, что и предполагалось

Кстати, вот еще есть такой HTML отчет
`hw5/HomeTask05Java/build/reports/problems/problems-report.html`

Дальше вот что написал мне AI:

Что было исправлено в профилировании

- `NullPointerException` в `Client.deposit(...)`, если на итерации `register(...)` возвращал `null`.

Исправления:

1. `src/main/ru/hse/profiling/ServerProfilingLoadTest.java`

- добавлена защита от `null` после `register(...)` (итерация учитывается как `failed`, тест не падает),

- логин сделан уникальным для каждого запуска (`runId`), чтобы не конфликтовать с уже созданными пользователями в БД.

2. `src/main/ru/hse/client/Client.java`

- в `register(...)` добавлена обработка `INCORRECT_RESPONSE` как исключения.

3. `src/main/ru/hse/client/AccountManager.java`

- в `register(...)` добавлена корректная обработка `ALREADY_INITIATED`.

Проверка правил серверной части

Проверено:

1. После logout очищаются in-memory структуры сессий (`activeSessions`, `sessionEnabled`).
2. После logout данные аккаунта остаются только в БД.
3. При остановке сервера закрывается пул БД (`HikariDataSource`) и останавливается web-server.